Mizore

Software Architecture Document

Architecture description of the current repository implementation

Document status	Draft	**Version**	1.0
System	Mizore	**Source basis**	Current checked-in code

Contents

1 Introduction **2**

1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 Audience . . . 2
1.4 Conventions . . . 2

2 Architecture Drivers **2**

2.1 Stakeholders and concerns . . . 2
2.2 Key use cases . . . 2
2.3 Constraints and assumptions . . . 3

3 System Context and Container View **3**

4 Architecture Views **3**

4.1 Logical and component view . . . 3
4.2 Runtime view . . . 4
4.3 Deployment view . . . 4
4.4 Data view . . . 5
4.5 Interface view . . . 5

5 Cross-Cutting Concerns **5**

5.1 Authentication and authorization . . . 5
5.2 File-path boundary . . . 5
5.3 Persistence and migration . . . 5

6 Architecture Decisions and Rationale **6**

7 Quality Attribute Assessment **6**

8 Risks and Open Issues **6**

9 Glossary **7**

10 References **7**

1 Introduction

1.1 Purpose

This document describes Mizore's architecture: its system boundary, runtime components, responsibility allocation, data stores, interfaces, deployment shape, and notable constraints. It is a description of the current implementation, not a feature specification or a proposal for an unimplemented target architecture.

1.2 Scope

Mizore is a browser-based shared-file navigator. Users sign in, manage saved directory paths, and browse or open files exposed by the backend. The architecture in scope includes the Next.js frontend, Fastify API, PostgreSQL database, and shared filesystem used by the backend.

1.3 Audience

This document is intended for developers, maintainers, and operators who need to locate responsibilities, understand service interactions, or assess the impact of a change.

1.4 Conventions

The architecture views are implementation-oriented. Solid arrows indicate runtime requests or data access. A dashed boundary indicates services grouped by Docker Compose. Statements marked as current behavior describe the checked-in code and may include known gaps.

2 Architecture Drivers

2.1 Stakeholders and concerns

Stakeholder	Primary architecture concern
Browser user	Sign in, access saved paths, and browse files through the web UI.
Owner or admin	Manage users and perform administrative operations.
Developer or maintainer	Change UI, API, persistence, or file handling without crossing layer boundaries accidentally.
Operator	Configure service ports, database connection, JWT secrets, and shared storage for deployment.

2.2 Key use cases

1. Authenticate with username and password and restore a client session.
2. Retrieve, create, update, and delete a user's saved paths.
3. List a directory and retrieve a file from the configured shared filesystem.

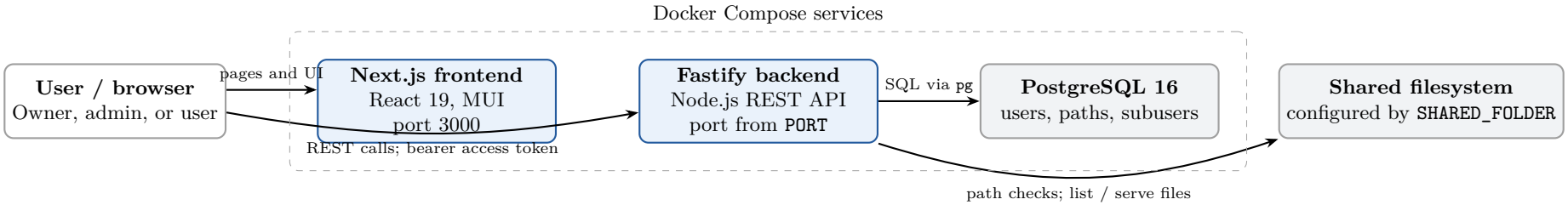
4. Perform user-administration operations subject to the configured role checks.

2.3 Constraints and assumptions

- The frontend and backend are separate services; browser API requests use `NEXT_PUBLIC_API_URL`.
- PostgreSQL stores application records. File contents remain on the shared filesystem.
- The checked-in deployment uses Docker Compose, PostgreSQL 16, and Node.js 20 images.
- No formal service-level objectives, capacity targets, or availability targets are stated in the repository documentation.

3 System Context and Container View

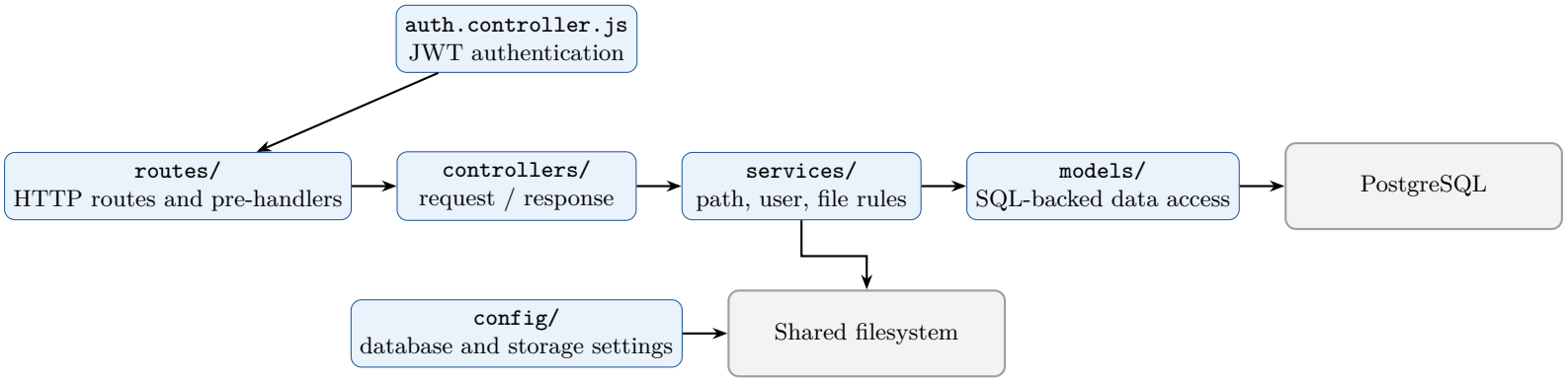
Mizore is used through a browser. The browser obtains the UI from Next.js and sends API requests to Fastify. Fastify reads and writes PostgreSQL records and accesses files beneath its configured shared-folder root. The database and shared filesystem are resources used by the application; neither is a separate user-facing service.



4 Architecture Views

4.1 Logical and component view

The backend follows a route–controller–service–model organization. Route modules map HTTP paths and attach pre-handlers; controllers translate requests and responses; services implement operations; models issue parameterized SQL queries. Authentication is implemented in `auth.controller.js` and is attached to routes that require it. The frontend separates pages, reusable components, and shared client state.



Component	Location and responsibility	Boundary
Frontend pages	`nextjs/src/app/`; login, user creation, admin, settings, and file viewer.	UI and page composition; API calls are made from the browser.
Frontend shared state	`nextjs/src/context/` and `components/`; session, path, current-folder state, and sidebar controls.	App context owns client-side authentication and path state.
HTTP API	`node_fastify/src/routes/` and `controllers/`; auth, user, path, and file endpoints.	Routes define middleware; controllers handle HTTP-level concerns.
Application logic	`node_fastify/src/services/`; path, user, and file operations.	Coordinates data access and filesystem operations.
Persistence and configuration	`node_fastify/src/models/` and `config/`.	Parameterized SQL, connection pool, and shared-folder root.
Database schema	`postgres/init.sql` and `postgres/migrations/`.	Initialization applies to an empty volume; existing databases require migrations.

4.2 Runtime view

Authentication flow. The browser posts credentials to `/api/login`. The API loads the user record, compares the password hash, and returns a user object and access/refresh JWTs. The client stores the tokens in local storage and sends the access token as a bearer token on authenticated requests. On session restoration, the client calls `/api/me`.

Saved-path flow. The client calls the authenticated `/api/paths` endpoints. Path creation resolves the submitted relative directory beneath `SHARED_FOLDER`, checks that it exists and is a directory, then stores its metadata in PostgreSQL.

File-browsing flow. The client requests directory entries through `/api/list/:file` and file content through `/api/files/:filename`. The backend uses the shared filesystem for these operations; file contents are not stored in PostgreSQL.

4.3 Deployment view

The Compose deployment contains frontend, backend, and PostgreSQL services. PostgreSQL uses the `postgres_data` named volume and initializes from `postgres/init.sql`; the backend waits for the database health check. The backend image starts `node server.js`; the frontend uses a multi-stage Next.js build and serves on port 3000. The backend's file root is intended to be supplied through `SHARED_FOLDER`.

4.4 Data view

Store / entity	Data	Relationship / use
`users`	ID, username, password hash, role, creation time.	Account records; roles include values used by route checks, such as `admin` and `owner`.
`subUsers`	ID, name, role, parent user ID, creation time.	Subordinate records associated with a user.
`paths`	ID, name, path, user ID, `allowed_subpath`.	Saved directory metadata associated with a user.
Shared filesystem	Directory entries and file bytes.	Read by the backend; root is configured separately from database records.

4.5 Interface view

Interface	Examples	Consumer / provider
Browser–frontend	HTTP pages and client-side UI.	Browser consumes the Next.js frontend.
Browser–API	`/api/login`, `/api/me`, `/api/paths`, `/api/list/:file`, `/api/files/:filename`.	Browser calls Fastify; `NEXT_PUBLIC_API_URL` configures the API base URL.
API–database	PostgreSQL protocol through the `pg` pool.	Fastify queries PostgreSQL using parameterized SQL.
API–filesystem	Directory and file operations under `SHARED_FOLDER`.	Fastify accesses the mounted shared directory.

5 Cross-Cutting Concerns

5.1 Authentication and authorization

Access tokens are signed as JWTs with a one-hour expiry; refresh tokens have a seven-day expiry. Path and current-user routes install authentication checks. User-administration routes require `admin` or `owner`. Token verification is performed by the backend.

5.2 File-path boundary

Path creation resolves a submitted relative path beneath the configured shared root and verifies that it is an existing directory. Listing and static file serving are implemented through separate routes, so they must be assessed independently when changing filesystem behavior.

5.3 Persistence and migration

PostgreSQL schema initialization is performed by `postgres/init.sql` when the data volume is first created. The `allowed_subpath` column is also represented by a migration for databases that already exist.

6 Architecture Decisions and Rationale

Decision reflected in code	Rationale or observed consequence
Separate frontend and API services	UI and API can be built and started independently; browser API access depends on a correct public API URL.
PostgreSQL for application records	Relational user and saved-path data is kept separately from the contents being browsed.
Shared filesystem for files	The application does not copy file contents into the database; the backend needs access to a configured shared root.
Layered Fastify backend	Routes, controllers, services, and models have distinct HTTP, application, and persistence responsibilities.
Bearer JWT session	The client can authenticate API requests without a server-side session store; tokens are currently stored in browser local storage.

7 Quality Attribute Assessment

The repository does not define measurable quality-attribute targets. The following table records the design mechanisms and the current evidence or limitation rather than claiming that a target has been met.

Attribute	Current mechanism	Assessment / evidence
Security	Password hashing, bearer JWT authentication, role checks, path-root validation.	File-list and file-download routes do not attach authentication pre-handlers in the current route definitions.
Data durability	PostgreSQL named volume.	Database data persists across container recreation; initialization scripts do not automatically migrate existing volumes.
Modifiability	Layered backend and separate frontend/backend projects.	Responsibilities have defined locations; API and storage configuration remain deployment-sensitive.
Operability	Fastify logging and PostgreSQL health check.	No centralized monitoring, backup policy, or formal availability target is specified in the repository.

8 Risks and Open Issues

- **Frontend API URL:** the checked-in Compose value is `http://localhost:3000`, while Fastify is a separate service on its configured port. Configure a browser-reachable backend URL for the deployment.
- **Shared-folder mount:** Compose mounts the host's `/home/yuki` at that container path, while the backend defaults `SHARED_FOLDER` to `/files`. Unless runtime configuration changes, these paths do not refer to the same directory.
- **File-route authorization:** `/api/list/:file` and `/api/files/:filename` do not install authentication pre-handlers, even though the browser may send an access token.
- **Refresh behavior:** the client stores a refresh token, but the current route set does not expose a refresh endpoint.

- **Path update validation:** path creation checks containment and directory existence; the update operation currently writes path metadata without performing those same checks.

9 Glossary

API	Application Programming Interface; in this system, the Fastify HTTP endpoints.
JWT	JSON Web Token used by the API to authenticate bearer-token requests.
Shared root	Filesystem directory configured for backend file operations through `SHARED_FOLDER`.
Saved path	A PostgreSQL record identifying a directory available to a user in the UI.

10 References

- `README.md` — project overview and development instructions.
- `DOCKER_DEPLOYMENT.md` — deployment configuration and migration notes.
- `docker-compose.yml` — checked-in service topology and mounts.
- `node_fastify/src/` — API routes, controllers, services, models, and configuration.
- `nextjs/src/` — frontend pages, components, and client-side application state.
- `postgres/init.sql` and `postgres/migrations/` — database schema and migration.